

JAVA INTERVIEW PREPARATION

CHAPTER 52

RELATIONAL SCHEMA EVOLUTION AND ZERO-DOWNTIME MIGRATIONS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Relational Schema Evolution: Versioned Migrations and Zero-Downtime Compatibility

Senior / Lead Java Developer Reading Chapter

A database schema is a shared runtime contract between deployed application versions, operational tools, data pipelines, and stored history. In a rolling deployment, old and new application instances use the same database simultaneously. Safe migration therefore requires compatibility across time, not just syntactically correct DDL.

Migration Files Are Immutable History

Versioned migrations record ordered schema and data changes in source control.

```
V047__add_order_external_reference.sql
V048__create_unique_reference_index.sql
V049__make_reference_required.sql
```

After a migration is applied to a shared environment, do not edit it casually. Tools such as Flyway store migration identity and checksum in a schema-history table and validate local files against applied history. Create a new corrective migration instead.

Changing old history makes environments disagree and destroys reproducibility. Repair commands alter migration metadata and should be controlled operational actions, not routine ways to hide unexpected differences.

Schema Generation Is Not Production Migration

ORM schema generation is useful for experiments and some isolated tests. It does not provide reviewed, ordered, restartable production evolution, data backfill, online-index strategy, or audit.

Production should normally validate mappings against a schema created by the same versioned migrations used in deployment. Tests that let Hibernate create tables can pass while the real migration is missing a constraint, type, index, or default.

Keep JPA mapping and migration ownership aligned. A mapping change without a compatible migration is incomplete work.

Expand and Contract

Zero-downtime changes are usually multi-release transitions.

```
EXPAND:  add new compatible schema capability
MIGRATE: deploy code and backfill data
CONTRACT: remove old behavior after all users have moved
```

For a column rename from `name` to `display_name`:

1. add nullable display_name
2. deploy code that writes both; reads display_name with fallback to name
3. backfill existing rows in bounded chunks
4. verify parity and switch reads
5. stop old writes
6. later enforce constraints and drop name

A direct rename can break old instances during a rolling deployment. The transitional duplication is deliberate compatibility debt with an owner and removal date.

Adding a Required Column Safely

Adding **NOT NULL** without a value can fail for existing rows or lock/rewrite a large table. Use phases:

```
add nullable column
-> deploy writers
-> backfill old rows
-> verify no nulls
-> add or validate constraint
-> make application assumption official
```

Database versions differ in whether adding a constant default rewrites the table. Verify target-engine behavior rather than relying on an old rule. Even metadata-only DDL can wait behind long transactions or block concurrent statements while acquiring its lock.

Backfill Engineering

A backfill is a production workload. It consumes connections, CPU, I/O, locks, log/WAL, replica bandwidth, and cache.

```
update customer
set normalized_email = lower(email)
where id > :lastId
  and normalized_email is null
order by id
limit :chunk;
```

Exact syntax varies, but the principles are bounded chunks, deterministic progress, idempotency, throttling, and resumability. Commit each chunk according to recovery needs and store a checkpoint or derive progress safely.

Monitor application latency, lock wait, replica lag, dead tuples or undo, transaction-log growth, and remaining rows. Pause automatically when service objectives are threatened. A one-shot update of hundreds of millions of rows is rarely operationally responsible.

Dual Write and Backfill Races

While backfill runs, live traffic continues writing. A backfill can overwrite a newer application value if it unconditionally derives from old data.

Use a predicate such as `new_column is null`, a version check, trigger, or explicit ownership protocol. Define which source is authoritative during every phase.

Application dual write is easier to observe in code but every writer must be upgraded. Database triggers cover legacy writers but add hidden behavior and lifecycle management. Choose deliberately and remove transitional mechanisms after contraction.

Online Index Creation

Building an index scans data and can consume substantial resources. Normal builds may block writes or wait for locks. Online or concurrent modes reduce blocking but take longer, may leave invalid artifacts after failure, and still generate I/O and replication lag.

```
create index safely
-> monitor build and replica health
-> verify valid index and expected plan
-> deploy query depending on it
-> remove redundant index later
```

Do not deploy code that requires an index before the index is complete across the target database fleet. An index is usually a performance dependency rather than correctness, but a unique index can also enforce an invariant and may fail because duplicates already exist.

Lock Planning Before DDL

Many DDL statements need a strong catalog or table lock, even when the actual metadata change is fast. A long-running transaction can block the DDL; once queued, the DDL can in turn block later ordinary queries, creating an outage chain.

```
old report transaction holds lock
-> ALTER TABLE waits
-> new application statements queue behind ALTER
```

Inspect active transaction age before migration, set deliberate lock timeouts, and fail quickly rather than waiting indefinitely in the request path. Retry during a controlled window after removing the blocker safely. Statement timeout and lock timeout serve different purposes.

Some teams run a no-op or lock-only rehearsal to measure acquisition behavior before the real change. Always test on the target engine and version because lock levels vary by operation.

Enum and Lookup Evolution

Database enum types, CHECK constraints, and lookup tables encode allowed values differently. Adding a value may be easy while removing or renaming it is difficult because old rows and old application instances still use it.

Use tolerant readers and additive values first. Deploy code that understands old and new values before producing the new value. Stop production, migrate stored rows, verify no old values remain, then remove old compatibility later.

Java enums can throw during deserialization when the database contains an unknown value. A rolling deployment must not let new writers emit values old readers cannot parse. Mapping stable database codes through an application converter can decouple Java constant names from durable representation.

Data Verification and Reconciliation

A completed migration is not proven correct by exit code alone. Define verification queries and invariants:

```
old non-null count equals new populated count
no rows violate new foreign key
old and new calculated totals match within rounding policy
all dual writes have equal versions
```

For huge tables, exact checks may be expensive. Use partitioned counts, checksums, sampled comparisons, and exception queries while understanding collision and sampling limits. Keep a final exact constraint where the database can validate it.

Record verification evidence with the deployment. A backfill progress metric reaching 100 percent can still hide rows skipped by an incorrect predicate.

Destructive Data Retention Changes

Dropping columns and tables removes rollback evidence and may conflict with legal retention or deletion policy. Confirm downstream analytics, backups, replicas, CDC connectors, and support tools no longer depend on the data.

Mark a column deprecated, stop writes, observe reads where possible, archive or export only when policy permits, then drop in a later release. Backups retain deleted data according to their own lifecycle; a schema drop is not immediate erasure from every copy.

Database Views as Compatibility Bridges

A view can temporarily preserve an old read shape while base tables evolve. It is useful for read-only legacy consumers, reporting tools, or gradual table splits.

Writable compatibility views and triggers add complexity and database-specific semantics. Use them only with a clear owner and retirement plan. A permanent compatibility layer can conceal an incomplete migration and limit optimizer choices.

Adding Uniqueness

Before adding a unique constraint, profile duplicates and decide resolution. Deduplicate with auditable business rules; do not silently delete arbitrary rows.

For a large table, some databases allow building a unique index online and attaching a constraint afterward. Concurrent writes must be included correctly by the engine. Test failure cleanup and retry.

Keep the application conflict handling even after the constraint exists. The pre-check improves feedback; the constraint closes the race.

Foreign Keys on Existing Data

Adding a foreign key can scan the child table and lock or load the database. Existing orphan rows cause failure.

Some engines support adding a constraint as not yet validated, enforcing it for new writes, then validating existing data separately. This separates lock-sensitive definition from long validation. Database-specific semantics must be verified.

Index child foreign-key columns when join and parent-delete behavior require it. The constraint alone does not universally create that index.

Type Changes

Changing column type can require a table rewrite and exclusive lock, and values may not convert safely. For risky changes use a new column:

```
add amount_minor_units bigint
dual write from application
backfill and validate conversion
switch reads
remove old decimal representation later
```

Define rounding, overflow, timezone, collation, and null semantics before migration. A structurally successful conversion can still corrupt business meaning.

Table Splits and Merges

Splitting one table into two or merging schemas needs identity and update ownership during transition. Create the new structure, populate it, keep writes synchronized, compare counts and checksums, switch reads gradually, then retire old state.

Avoid cross-database dual write when possible. If the new owner is another service, use an outbox or change-data capture plus reconciliation, and expect eventual convergence rather than pretending one local transaction covers both databases.

Application Compatibility Matrix

Document which application versions work with which schema phases.

	schema old	expanded	contracted
app old	yes	yes	no
app transition	no	yes	yes
app new	no	yes	yes

The exact matrix varies, but it exposes rollback limits. Once contraction occurs, rolling back to old code may be impossible. Deployment automation should know the minimum schema version and reject unsafe combinations.

Forward Fix Versus Rollback

Database rollback is not equivalent to application traffic rollback. Destructive DDL and data transformations may be irreversible, and new code may have written values old code cannot understand.

Prefer backward-compatible migrations and forward fixes. Before deployment define rollback boundaries: traffic can return to version A until step X; after step Y, only a forward migration is safe.

Backup restore is disaster recovery, not a routine migration rollback. Restoring one database to an earlier time can conflict with messages, external payments, and other services that continued processing.

Transactional DDL Varies

Some databases support transactional DDL broadly; others implicitly commit around DDL or exclude particular operations. Migration tools can wrap scripts in transactions only when the engine supports it.

Test a failed migration on the actual database. Know whether partial objects remain, how schema history records failure, and what cleanup or repair is required. Do not assume a failed script automatically returns the schema to its prior state.

Migration Ownership in a Cluster

Multiple application replicas starting simultaneously must not race schema changes. Migration tools generally use database locking, but running heavy migrations inside every application startup still couples service availability to schema work.

For substantial production changes, use a controlled migration job or pipeline with identity, timeout, logs, approval, and observability. Application startup can validate expected schema version and fail clearly if incompatible.

Separate migration credentials from runtime credentials. The application usually needs DML permissions; DDL privileges broaden blast radius. Short-lived deployment identity improves control.

Validation and Drift

Validate checksums and applied migration order in CI and before deployment. Detect manual production drift through schema comparison or catalog inspection. An emergency manual change must become a versioned migration immediately so future environments reproduce it.

Do not expose migration credentials or SQL values in logs. Review scripts for destructive operations, full-table rewrites, unbounded updates, missing predicates, and lock behavior.

Multi-Tenant Schemas

Schema-per-tenant creates migration fan-out. Track version per tenant, limit parallelism, retry safely, and expose lagging tenants. One failure should not leave the fleet unobservable.

Shared-table tenancy avoids DDL fan-out but makes every migration affect the whole dataset. Partitioning and row-level security add their own compatibility needs. Choose tenancy partly from operational migration cost, not only logical isolation.

Testing Migrations

Test from a production-like previous schema, not only from an empty database. Apply the full history to create a clean environment, but also test the incremental path from every supported release baseline.

Verify:

- Old application works during expand.
- Transition code reads old and new data.
- Backfill resumes and does not overwrite live writes.
- Constraints validate after data migration.
- New application works before contraction.
- Rollback boundaries behave as documented.
- Query plans and latency remain acceptable.

Use realistic volume to reveal lock and rewrite behavior. A migration succeeding on ten rows proves little about a billion-row table.

Production Observability

Record migration version, start and completion, statement phase, rows processed, lock wait, database load, replica lag, and failure details. Backfills need progress and estimated remaining work without high-cardinality labels.

During deployment correlate application errors with schema phase. A surge in "column not found" is compatibility failure; increased latency may be index build or backfill pressure.

Keep a runbook for stopping backfill, cancelling DDL safely, cleaning invalid indexes, resolving failed history, and deciding forward fix versus rollback.

Common Senior-Level Traps

Traps include editing an applied migration, renaming or dropping a column in one release, adding NOT NULL before writers and backfill, running unbounded updates, depending on an index before it is valid, and assuming all DDL rolls back.

Other traps include application and migration sharing powerful credentials, every replica running long migrations at startup, no compatibility matrix, and treating backup restore as a safe local rollback.

A Strong Interview Answer

I treat schema as a versioned runtime contract shared by old and new application versions. Applied migrations are immutable history. Most zero-downtime changes use expand, migrate, and contract: add compatible structures, deploy dual-compatible code, backfill in bounded resumable chunks, verify, switch reads and writes, then remove old structures later. I test lock and transactional-DDL behavior on the target engine, build large indexes safely, and add constraints only after data is clean. A compatibility matrix defines rollback limits. Migrations run through controlled identity and one owner, with progress, load, replica-lag, and drift monitoring.

Interview-Ready Summary

Safe migrations are distributed deployments across time. DDL correctness is not enough; old code, new code, existing data, live writes, replicas, and operational rollback must coexist.

Versioned immutable history, expand-and-contract sequencing, bounded backfills, target-engine testing, and explicit compatibility evidence turn schema change into a controlled production workflow.

Revision Notes

- Database schema is a runtime contract across application versions.
- Applied versioned migrations should remain immutable.
- Validation detects missing, changed, or reordered migration history.
- ORM schema generation is not a production migration strategy.
- Expand, migrate, and contract across separate deployment phases.
- Direct rename or drop can break old rolling instances.
- Add required columns through nullable, write, backfill, verify, constrain.
- Backfills must be bounded, resumable, idempotent, and throttleable.
- Protect live writes from backfill overwrite races.
- Online index builds still consume resources and have failure states.
- Clean duplicates before enforcing uniqueness.
- Foreign-key validation can be a large operational scan.
- Risky type changes often use a new column and dual compatibility.
- Schema compatibility defines application rollback limits.
- Traffic rollback does not reverse data or DDL.
- Transactional DDL support differs by database and operation.
- Use one controlled migration owner in clustered deployments.
- Separate DDL migration identity from runtime DML identity.
- Detect and version emergency manual drift.
- Test incremental migration from supported previous releases.
- Use realistic volume to reveal locking, rewrite, and log impact.
- Observe progress, database load, lock wait, and replica lag.